LECTURE 2

HTML Foundations & Semantic Structure

From the first index.html file to valid, meaningful HTML

COURSE	INSTRUCTOR
Web Development Bootcamp **Cohort 1 (Front-End)**	**Newton Katete**

Why this lecture comes next.

Lecture one explained how networks, DNS, HTTP, browsers and servers work together to deliver a webpage. We now move from the infrastructure around the page to the markup that gives the page its structure and meaning.

CORE IDEA

We will learn HTML semantically from the beginning: choose elements because of what the content means, not because of how a browser happens to display them.

Learning outcomes

- Explain what HTML is and why Semantic HTML matters.
- Create a correctly named HTML project and understand why index.html is commonly the entry document.
- Distinguish tags, elements, content, and attributes.
- Identify the <head> and <body> areas of an HTML document.
- Use headings, paragraphs, and major semantic layout elements appropriately.
- Use the lang attribute correctly on the root <html> element.
- Use the W3C validation service to locate and correct markup errors.
- Understand why <!DOCTYPE html>, character encoding, viewport metadata, author metadata and favicons belong in a standards-ready document.

1. What Is HTML?

HTML stands for HyperText Markup Language. It is the markup language used to describe the structure and meaning of content in a web document. The browser reads the HTML and builds an internal representation of the page before CSS styles it and JavaScript adds behavior.

HTML IS NOT THE DESIGN LAYER

HTML answers questions such as: What is the main heading? Which content is a paragraph? Which area is the main content? CSS later answers questions about color, spacing, layout and appearance.

1.1 What does “Semantic HTML” mean?

Semantic HTML means selecting an element according to the meaning or role of its content. A semantic element gives useful information about the structure of the page to developers, browsers, and assistive technologies.

Approach	Example	What it communicates
Generic	...	A container, but not what the content means.
Semantic	<header> ... </header>	Introductory content for a page or section.
Semantic	<main> ... </main>	The dominant main content of the document.
Semantic	<footer> ... </footer>	Footer information for a page or section.

1.2 Why use Semantic HTML?

- Accessibility: semantic landmarks and headings help assistive technologies understand and navigate the page.
- Readable code: another developer can understand the document structure more quickly.
- Maintainability: meaningful structure is easier to update as a project grows.
- Search and content understanding: meaningful markup can help user agents and search systems interpret page structure.
- Cleaner foundations for CSS and JavaScript: meaningful elements make later styling and scripting easier to reason about.

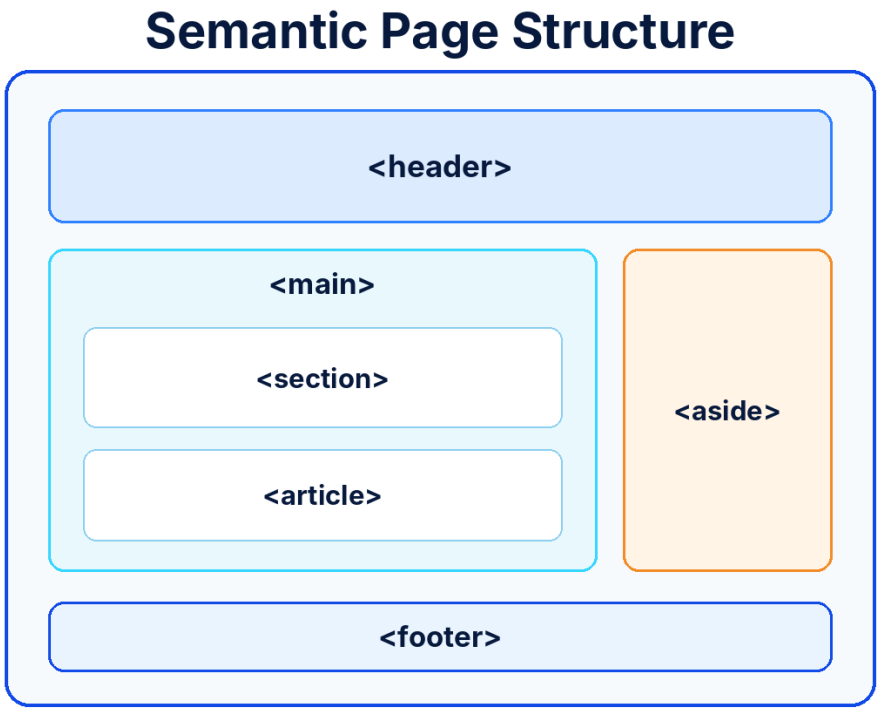


Figure 1. A simplified semantic page structure.

2. Start With the Project Folder and index.html

A web project should be organised before you begin writing many pages. Create one folder for the website, then place the project files inside it. For this lecture we will begin with only one HTML file.

2.1 Why is index.html important?

The filename index.html is a long-standing web convention for the default document in a folder. When a user requests a directory such as https://example.com/ rather than naming a specific file, many web servers are configured to look for an index file and serve it automatically. The exact default file is server-configurable, so index.html is a convention rather than a law of the Internet.

REMEMBER

For beginner front-end projects, use index.html as the home/entry page unless you have a specific reason not to. This also works naturally with static hosting services such as GitHub Pages.

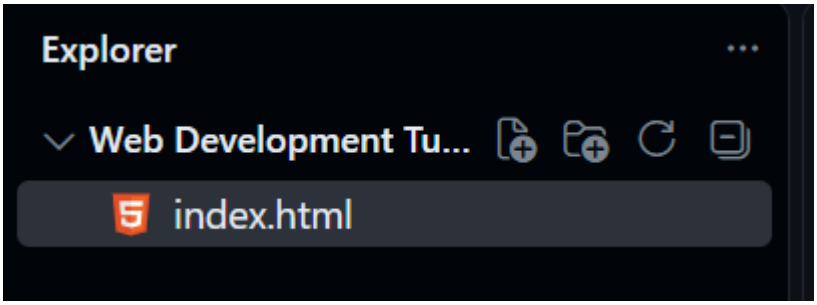
2.2 Good HTML file naming conventions

Good practice	Example	Reason
Use lowercase	about.html	Predictable across systems and servers.
Use hyphens between words	contact-us.html	Readable in URLs; avoids spaces.
Use meaningful names	services.html	A learner can tell what the page contains.
Keep the .html extension	portfolio.html	Clearly identifies the document type.
Use index.html for the entry page	index.html	Common default-document convention.
Avoid spaces	my-project.html	Spaces become awkward in URLs and file paths.
Avoid unnecessary symbols	student-profile.html	Simple names reduce path and portability problems.

AVOID

My Page.HTML, webpage 1.html, final-final!.html, NewPage (2).html. Prefer names such as index.html, about.html and student-profile.html.

Below is a screenshot of a folder that we will be using for this tutorial lecture series?



3. Tags, Elements and Content

Beginners often use the words tag and element as though they mean the same thing. They are closely related, but it is useful to distinguish them.

A tag is the markup syntax inside angle brackets, for example or . An element is the complete HTML unit: the opening tag, its content, and usually the closing tag.

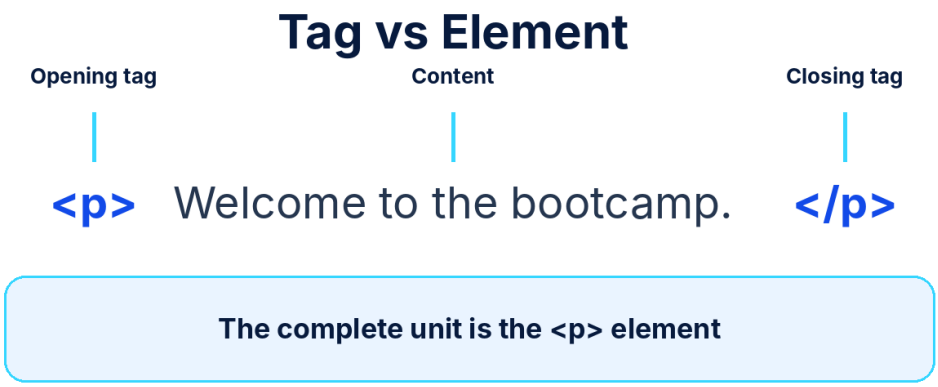


Figure 2. The tags are parts of the element.

3.1 Container elements and void elements

Most of the elements we use in this lecture are container elements: they have an opening tag and a closing tag with content between them. Some HTML elements are void elements and do not wrap content. Examples you will meet during the bootcamp include
, <hr>, <img>, <meta> and <link>.

3.2 Grouping HTML elements by purpose

Group	Examples	Main purpose
Document/root	<html>	Contains the complete HTML document.
Document information	<head>, <title>	Information about the page rather than the visible page content.
Visible page	<body>	Contains content presented to the user.
Semantic structure	<header>, <main>, <section>, <article>, <aside>, <footer>	Describes the major regions and meaning of the page.
Headings and text	<h1>–<h6>,	Organises headings and paragraph content.
Generic containers	,	Useful when no meaningful semantic element describes the content.
Void elements	 , <hr> (and later <meta>, <link>, <img>)	Elements that do not contain child content.

SEMANTIC RULE

Use the most meaningful element available. Do not replace every section of a page with just because is flexible.

4. Build the First HTML Page

We will begin with a deliberately minimal HTML document. At this stage, we will NOT add the doctype declaration or metadata elements yet. Later in the lecture, the validator will help us discover why a standards-ready page needs them.

Step 1 — Create the document structure.

```
<html>
<head>
    <title>My First Web Page</title>
</head>
<body>

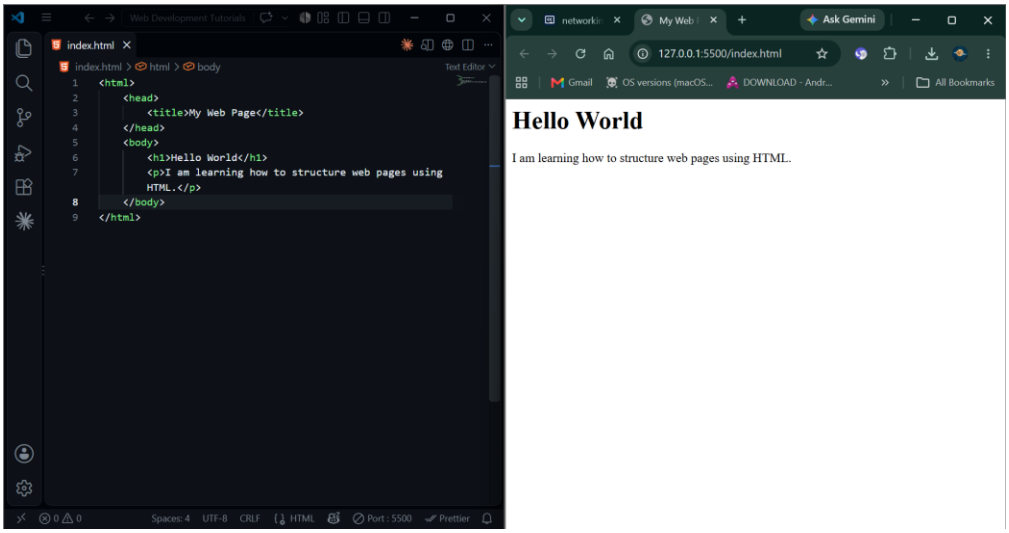
</body>
</html>
```

Step 2 — Add visible content to the body.

```
<html>
<head>
    <title>My First Web Page</title>
</head>
<body>
    <h1>Hello World</h1>
    <p>I am learning how to structure web pages with HTML. </p>
</body>
</html>
```

Step 3 — Save and open index.html in the browser.

1. Save the file as index.html inside the project folder.
2. Open the file in a web browser or use a local preview tool if one is available.
3. Observe that the <title> appears in the browser tab while <h1> and <p> appear inside the page.
4. Make a slight change in VS Code, save, then refresh the browser to see the update.



5. The Two Main Areas Inside the HTML Document

Inside the root <html> element, beginners should think about two major areas: the <head> and the <body>. They have different jobs.

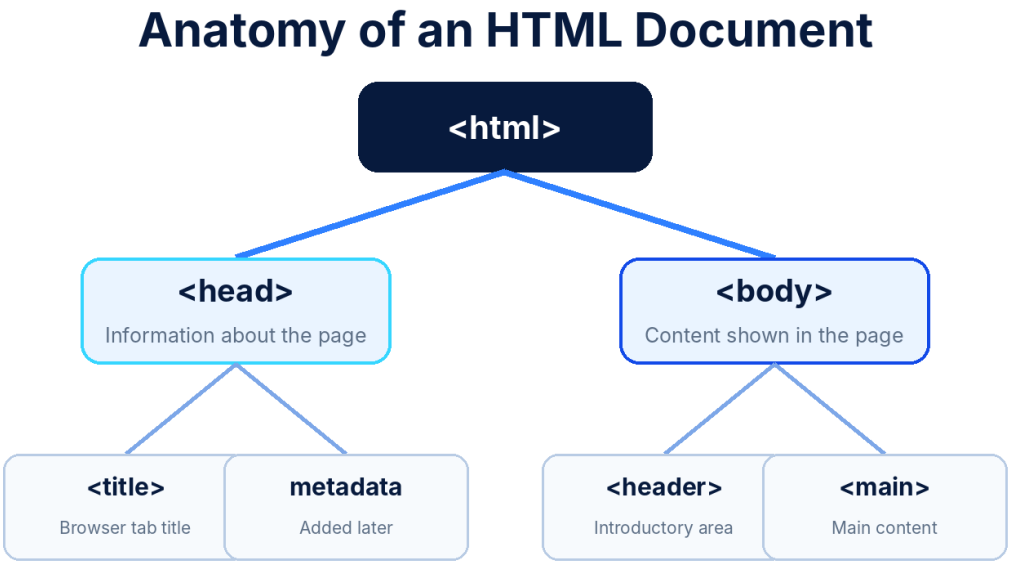


Figure 3. Simplified relationship between the root html element, the head, and the body.

5.1 The <head> area

The <head> contains information about the document. Most head content is not displayed as normal page content. For now, our only head element is <title>, which gives the document a title that browsers commonly show in the tab or window title.

```
<head>
    <title>My First Web Page</title>
</head>
```

5.2 The <body> area

The <body> contains the document content that the user interacts with or reads in the page. Headings, paragraphs and the main semantic regions of the page belong here.

```
<body>
    <h1>Welcome to My Website</h1>
    <p>I am learning Semantic HTML.</p>
</body>
```

Head	Body
Information about the document.	Visible/interactive document content.
Contains <title> and later metadata/resource links.	Contains headings, paragraphs and semantic page regions.
Not a place for normal visible page content.	Where the structure the user sees is described.

6. The Paragraph Element <p>

The <p> element represents a paragraph of text. Use a separate paragraph element for each distinct paragraph instead of inserting many line breaks to create visual spacing.

```
<p>HTML gives a webpage its structure and meaning.</p>
<p>CSS will later control how that structure looks.</p>
```

6.1 Paragraphs are semantic content

Using <p> tells the browser and other tools that the enclosed text is a paragraph. The browser normally collapses repeated whitespace in ordinary HTML text, so pressing the space bar many times is not a reliable layout method.

DO NOT USE HTML TO FAKE DESIGN

Do not add extra spaces, empty paragraphs or repeated
 elements just to push content around. Later, CSS will control spacing.

6.2 Headings and paragraphs work together

A heading introduces a topic; the paragraphs that follow explain it. Use heading levels to create a logical content hierarchy, not because a heading happens to look bigger or smaller.

```
<h1>My Student Profile</h1>

<h2>About Me</h2>
<p>I am learning front-end web development.</p>
<p>This page is structured using meaningful HTML elements.</p>
```

7. Major Semantic Structure Elements

Now we can replace a flat collection of headings and paragraphs with meaningful page regions. These elements describe the role of content, not its visual appearance.

Element	Meaning / typical role	Important note
<header>	Introductory content for a page or section.	Can contain a main heading, introductory text and later navigation.
<main>	The dominant content of the document.	A page should normally have one main landmark representing its primary content.
<section>	A thematic grouping of related content.	Usually makes sense with a heading.
<article>	Self-contained content that could stand independently.	Useful for posts, news items, cards, tutorials or independent entries.
<aside>	Related or supplementary content.	Should be related to surrounding or overall page content.
<footer>	Footer information for a page or section.	Often contains authorship, copyright or related information.

7.1 Non-semantic versus semantic structure

A generic structure — possible, but the roles are unclear

```
<div>
  <div>My Website</div>
</div>
<div>
  <div>About Me</div>
  <div>I am learning HTML.</div>
</div>
```

A semantic structure — the roles are clearer

```
<header>
  <h1>My Website</h1>
</header>

<main>
  <section>
    <h2>About Me</h2>
    <p>I am learning HTML.</p>
  </section>
</main>
```

7.2 Build the first semantic page

```
<html>
<head>
  <title>My Student Profile</title>
</head>
<body>
  <header>
    <h1>My Student Profile</h1>
    <p>Welcome to my first semantic webpage.</p>
  </header>

  <main>
    <section>
      <h2>About Me</h2>
      <p>I am learning how the Web works and how HTML structures content.</p>
    </section>

    <section>
      <h2>My Learning Goal</h2>
      <p>My goal is to build clear and accessible websites.</p>
    </section>
  </main>

  <footer>
    <p>Web Development Bootcamp 2026</p>
  </footer>
</body>
</html>
```

8. Attributes and the lang Attribute

An attribute adds extra information or configuration to an HTML element. Attributes are written inside the opening tag. Many attributes use a name/value pair.

General attribute pattern

<element attribute="value">Content</element>

For example, the lang attribute on the root <html> element declares the language of the document content.

```
<html lang="en">
    ...
</html>
```

8.1 Why lang="en" matters

- Screen readers can choose more appropriate pronunciation rules for the document language.
- Browsers and language-aware tools can interpret the page more accurately.
- Translation, spelling and other language-sensitive systems receive a useful language signal.
- It is good accessibility practice to identify the main language of the document.

LANGUAGE TAG

Use lang="en" when the page is primarily English. More specific BCP 47 language tags can be used when needed, but beginners should first learn to declare the correct primary language.

8.2 Update the page root

```
<html lang="en">
<head>
    <title>My Student Profile</title>
</head>
<body>
    ...
</body>
</html>
```

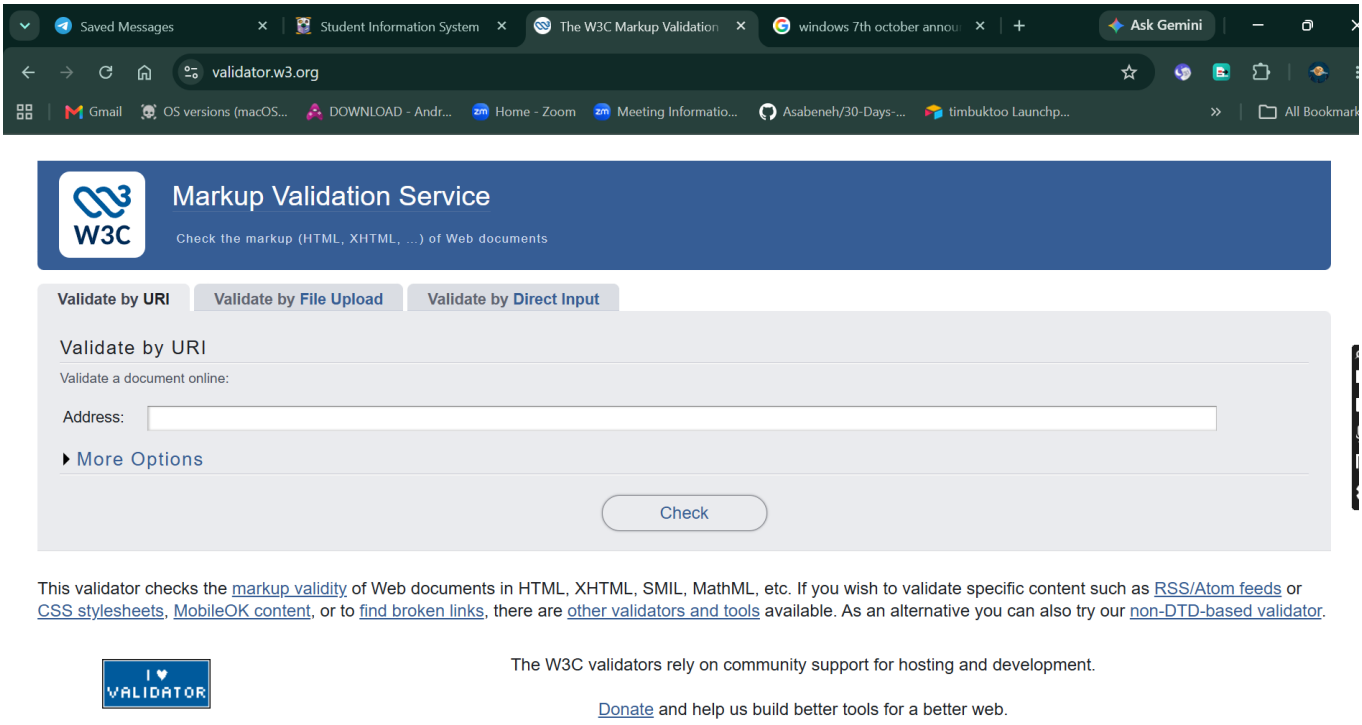
9. Finding Errors With the W3C Markup Validation Service

Browsers are forgiving: they often display a page even when its markup contains mistakes. A validator helps you check whether the markup follows HTML rules and highlights errors that may be easy to miss by simply looking at the page.

The W3C Markup Validation Service can validate by web address, file upload or direct input. For a local beginner project, file upload or direct input is usually the simplest approach.

VALIDATOR

W3C Markup Validation Service: https://validator.w3.org/
Modern HTML checking is also available through the W3C Nu HTML Checker.



9.1 Start with a deliberately broken example

```
<html lang="en">
<head>
  <title>My Student Profile</title>
</head>
<body>
  <header>
    <h1>My Student Profile</h2>
  </header>

  <main>
    <section>
      <h2>About Me</h2>
      <p>I am learning Semantic HTML.</p>
    </main>
  </section>
```

```
    <footer>
        <p>Web Development Bootcamp 2026</p>
    </footer>
</body>
</html>
```

This example contains mismatched heading tags and incorrect nesting: the <section> begins inside <main> but closes after </main>. A browser may still attempt to repair the structure, but that does not make the source correct.

9.2 Validation workflow

5. Save index.html.
6. Open https://validator.w3.org/.
7. Choose file upload or direct input.
8. Upload/paste the HTML and run the validation.
9. Read the messages carefully. Pay attention to the reported line/column and the element named in the message.
10. Correct one structural problem at a time in VS Code.
11. Save the file and validate again until the markup errors are resolved.

10. Making the Document Standards-Ready

Validation now gives us the right moment to introduce information that belongs at the top of a complete HTML document. These additions do not replace semantic body structure; they tell the browser and other tools important facts about the document itself.

10.1 <!DOCTYPE html> — the document type declaration

<!DOCTYPE html> is a declaration placed before the root <html> element. It is not an HTML element or a normal tag. In modern HTML, this short declaration tells browsers to interpret the document using standards mode rather than older compatibility behavior.

```
<!DOCTYPE html>
<html lang="en">
    ...
</html>
```

10.2 Character encoding — <meta charset="UTF-8">

Character encoding tells the browser how the bytes in the document should be interpreted as text. UTF-8 supports a very large range of characters and is the standard choice for modern web documents. Place the charset declaration near the beginning of the <head>.

```
<meta charset="UTF-8">
```

10.3 Viewport metadata

The viewport metadata helps mobile browsers size the page according to the device width. It becomes especially important when we begin responsive CSS.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

10.4 Author metadata

Author metadata can identify the author of the document. It is optional, but it demonstrates how name/content metadata can describe a page.

```
<meta name="author" content="Student Name">
```

10.5 Favicon

A favicon is the small site icon commonly displayed in a browser tab or bookmark. A favicon is NOT a <meta> tag. It is normally referenced using a <link> element inside the <head>.

```
<link rel="icon" href="favicon.png" type="image/png">
```

IMPORTANT DISTINCTION

<meta> and <link> are both void elements used in the head, but they have different purposes. The favicon is linked with <link rel="icon">; it is not declared with a meta tag.

11. The Complete Standards-Ready Page So Far

We can now combine the semantic structure we built earlier with the document declaration, language information and head metadata introduced through validation.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="author" content="Student Name">
    <title>My Student Profile</title>
    <link rel="icon" href="favicon.png" type="image/png">
</head>
<body>
    <header>
        <h1>My Student Profile</h1>
        <p>Welcome to my first semantic webpage.</p>
    </header>

    <main>
        <section>
            <h2>About Me</h2>
            <p>I am learning how the Web works and how HTML structures content.</p>
        </section>

        <section>
            <h2>My Learning Goal</h2>
            <p>My goal is to build clear and accessible websites.</p>
        </section>
    </main>

    <footer>
        <p>Web Development Bootcamp 2026</p>
    </footer>
</body>
</html>
```

11.1 Validate again

Run the updated file through the validator again. If there are remaining errors, use the reported lines and element names to locate the source of the problem. Validation is an iterative habit: write, save, test, validate, correct, and repeat.

12. Common Beginner Errors and a Debugging Checklist

Problem	Example / symptom	What to check
Wrong file extension	index.html.txt	Enable file extensions and ensure the file ends in .html.
Poor filename	My Web Page.HTML	Use lowercase, hyphenated names such as my-web-page.html.
Mismatched tags	Title	Opening and closing tag names must match.
Incorrect nesting	Close a parent before its child	Elements should close in the reverse order they were opened.
Visible content in <head>	Paragraph placed in head	Normal page content belongs in <body>.
No document language	<html> without lang	Add an appropriate lang attribute.
Using for everything	No meaningful landmarks	Prefer semantic elements when they describe the content.
Spacing with repeated 	Lots of manual line breaks	Use semantic paragraphs; CSS will later control layout and spacing.
Ignoring validator messages	Page “looks fine”	Validate the source even when the browser renders something.

12.1 Quick debugging routine

12. Read the browser output, but do not assume visual output proves the HTML is correct.
13. Check the exact line you recently edited.
14. Confirm opening and closing tags match.
15. Check nesting and indentation.
16. Confirm code is in the correct area: head or body.
17. Run the W3C validator and read the first relevant error message.
18. Fix, save and validate again.

14. Lecture Summary

- HTML describes the structure and meaning of web content.
- Semantic HTML chooses elements according to their role and meaning.
- index.html is commonly used as the default entry document for a website directory.
- Tags are pieces of markup; elements are complete HTML units.
- The <head> contains document information; the <body> contains the page content.
- The element represents a paragraph and should not be replaced with spacing tricks.
- Attributes add information/configuration to elements; lang identifies the document language.
- Validation helps identify markup and structural problems that a forgiving browser may hide.
- <!DOCTYPE html>, UTF-8 metadata, viewport metadata and other head information make the document more standards-ready.
- A favicon is linked using <link rel="icon"> inside the head; it is not a meta tag.

14.1 Key terms

Term	Meaning
HTML	HyperText Markup Language; the markup language used to structure web content.
Semantic HTML	HTML that uses elements according to the meaning/role of content.
Tag	Markup syntax such as or .
Element	The complete HTML unit formed by tags, content and attributes where applicable.
Attribute	Extra information/configuration written in an opening tag.
Head	Document information and resources that are not normal page content.
Body	The document content presented to the user.
DOCTYPE	A document type declaration used to trigger standards-mode HTML parsing.
Metadata	Data that describes or configures the document.
Validation	Checking markup against HTML rules/conformance requirements.